

# Intro to Circuits

Michael L. Littman

CS 22 2021

May 21st, 2021

# Overview

Basics

Addition

Expanded gates

# Logic and circuits

## Why AND/OR/NOT?

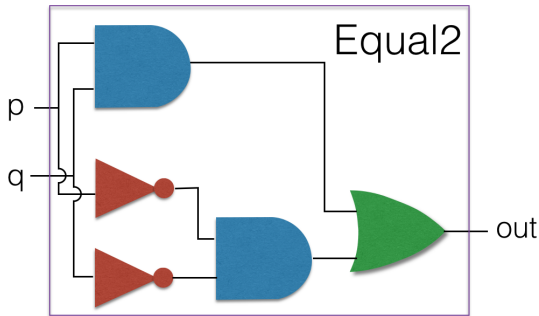
- ▶ Universal set.
- ▶ Already central to the study of logic.
- ▶ Can be implemented in physical hardware.

A *transistor* is roughly a voltage controlled switch and can be used to construct and/or/not. Voltage high: True. Voltage low: False.

To a first approximation, digital circuits are giant logic expressions. A computer is a giant logical expression whose outputs at one time step become the inputs at the next time step.

## Basic Circuit Components

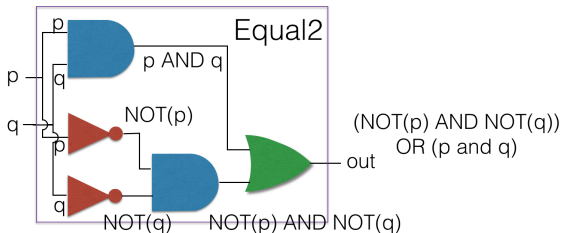
AND gates drawn as rectangles with a semicircular cap. OR gates similar, but concave rounded bottom and pointy top. NOT gates are triangles with a circle on the point.



How many and/or/nots?

## Circuit Flow

Wires show where values propagate to. We can label each wire with an expression relating the inputs of the circuit to the value propagating on that wire.

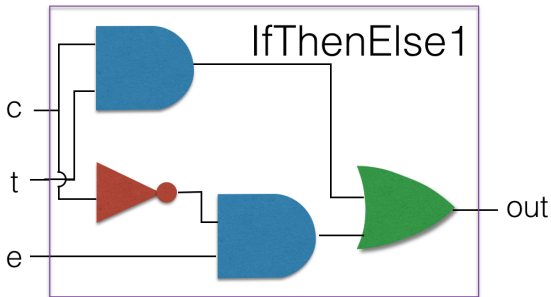


This circuit outputs  $(p \text{ AND } q) \text{ OR } (\text{NOT}(p) \text{ AND } \text{NOT}(q))$ .

We'll call this circuit "equal2" since it is testing the equality of 2 truth values or two bits. Another name?

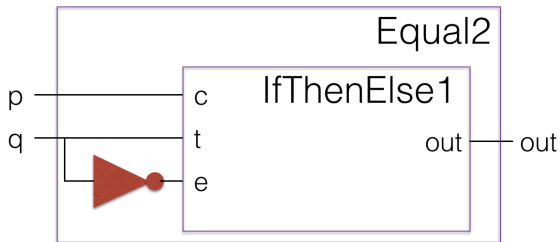
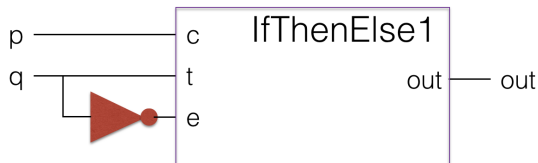
## IfThenElse1

Here's another useful circuit. It outputs either input  $t$  or input  $e$  depending on the value of condition  $c$ .



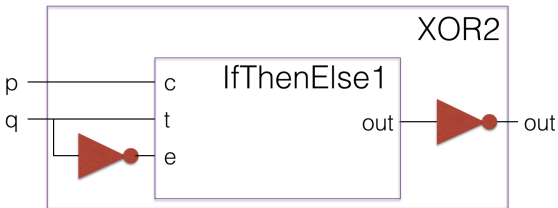
## Nested circuits

We can use `IfThenElse1` to make another circuit. What is it?



## Nested circuits 2

Here's another variant of IfThenElse1. What is it?





# Truth and numbers

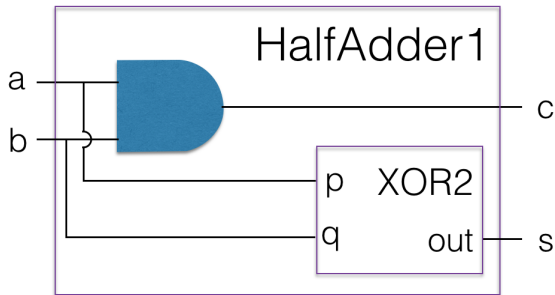
We can think of **T** as being 1 and **F** as being 0 and create circuits that compute on numbers. Let's add two bits.

|     |     | $c$     | $s$     |
|-----|-----|---------|---------|
| $a$ | $b$ | $a + b$ | $a + b$ |
| 0   | 0   | 0       | 0       |
| 0   | 1   | 0       | 1       |
| 1   | 0   | 0       | 1       |
| 1   | 1   | 1       | 0       |

$c = a \text{ AND } b$ .  $s = a \text{ XOR } b$ .

# Half adder

Here's a circuit for adding two bits.



## Adding two two-bit numbers

We can use the classic additional algorithm, modified to base 2.

Let's compute  $11 + 01$  (aka  $3 + 1$  in binary).

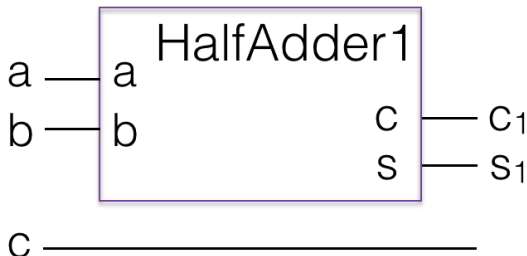
$$\begin{array}{r} 1 \ 1 \\ + \ 0 \ 1 \\ \hline \end{array}$$

$$\begin{array}{r} 1 \\ 1 \ 1 \\ + \ 0 \ 1 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ 1 \ 1 \\ + \ 0 \ 1 \\ \hline 1 \ 0 \ 0 \end{array}$$

## Full adder

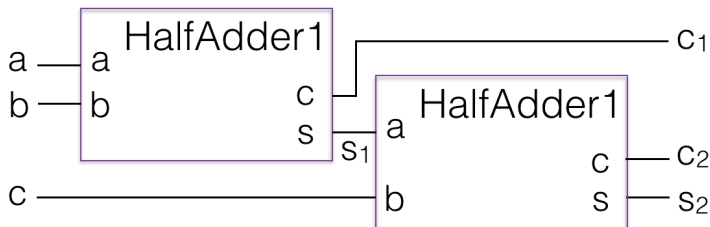
To handle this computation, we need to add *three* 1-bit numbers, not just two:  $a + b + c$ . Can add the first two bits to get a two-bit result.



Now, we need to add a two-bit number and a one bit number.

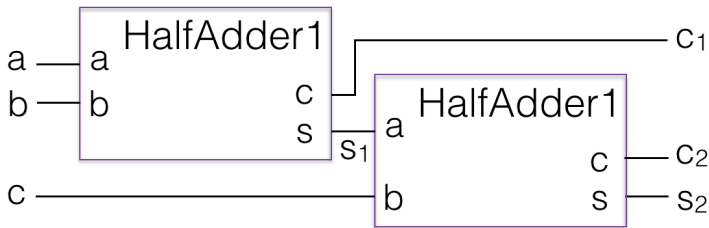
## Partial full adder

We have  $c_1 s_1 + c$ . Not quite clear what to do. Could add the low order bit  $s_1$  to  $c$ ...



The low order bit of the sum  $s_2$  is correct now. But, we have two carries,  $c_1$  and  $c_2$ , each worth two...

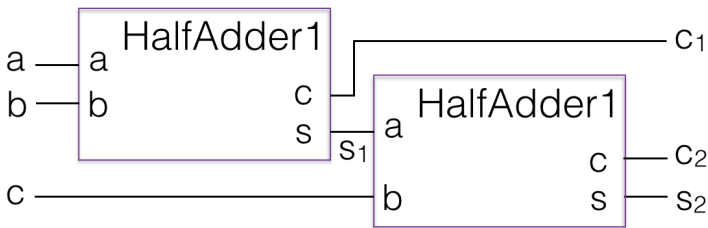
## Towards the full full adder



Need to add the two carries  $c_1$  and  $c_2$ . They are in the two's place. Could throw on another HalfAdder1. But:

- ▶ If  $a$  and  $b$  are both 1, their sum is 10. Thus, the carry  $c_1$  is a one, but the sum  $s_1$  is a zero. Adding in  $c$  won't cause a second carry in  $c_2$ .
- ▶ On the other hand, if at least one of  $a$  and  $b$  are less than 1, the bottom carry  $c_2$  might be 1, but the top one  $c_2$  is zero.

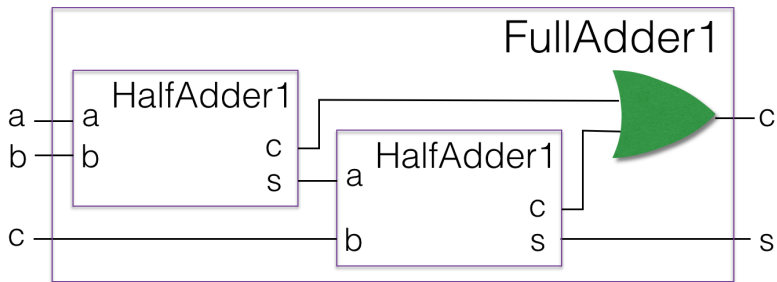
# Truth table for adding the carries



| $c_1$ | $c_2$ | $c_1 + c_2$ |
|-------|-------|-------------|
| 0     | 0     | 0           |
| 0     | 1     | 1           |
| 1     | 0     | 1           |
| 1     | 1     | —           |

What's the simplest gate that matches this truth table? OR!

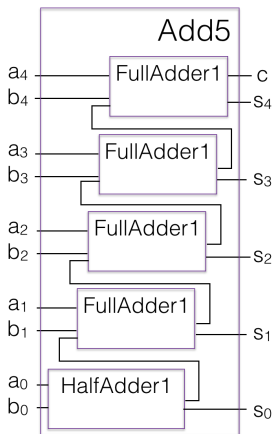
# Full full adder





## 5-bit-number adder

Now, we can do the fundamental operation for adding multi-bit values. Can daisy chain them to add multi-bit numbers.



## Deep circuits

Note that our circuits are getting kind of deep now.  
How deep?

- ▶ Add5 is 1 HalfAdder1 plus 4 FullAdder1 deep.
- ▶ FullAdder1 is 2 HalfAdder1 plus 1 OR deep.
- ▶ HalfAdder1 is 1 XOR2 deep (and 1 AND in parallel).
- ▶ XOR2 is 1 IfThenElse1 plus 2 NOT deep.
- ▶ IfThenElse1 is 1 NOT, 1 AND, and 1 OR deep.

That's 49 gates deep, if you work it out.

Time to compute grows with the depth of the circuit. Fortunately, there are better designs for adding.

## Negative numbers

We've defined addition in terms of logic gates. How about subtraction? We can compute subtraction via adding a negative number. What's a negative number in bits...?

$-x$  is a value such that  $x + (-x) = 0$ .

In computers, when we add two 5-bit numbers, we generally store a 5-bit answer, which means throwing away the carry. Thus, if we want the sum of two 5-bit numbers to result in a zero, we get that if they add to 32. So, to negate  $x$ , we can compute  $32 - x$  (or just 0 if  $x = 0$ ).

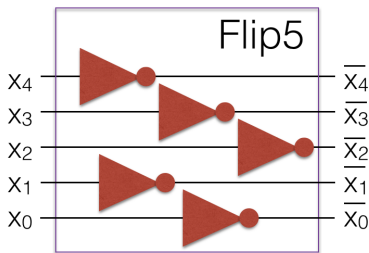
Example:  $-5 \rightarrow 27$ .

In binary:  $-00101 \rightarrow 11011$ .

Progress? Well, we have a way to represent a negative number, but computing it seems to require subtraction, which is what we were trying to figure out in the first place...

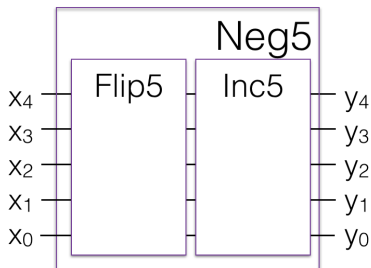
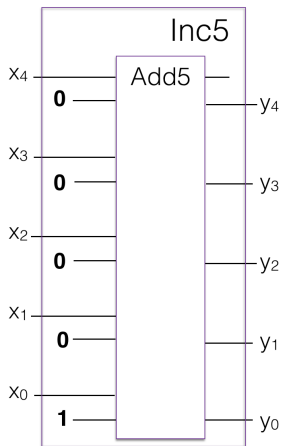
# Bit flips

Define  $\bar{x}$  to be the number  $x$  but with all its bits flipped.



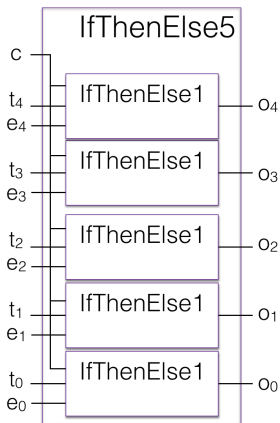
Note:  $x + \bar{x} = 2^5 - 1 = 31$  since the sum has every bit turned on. Thus,  $\bar{x} = 31 - x$ . If we add one to both sides, we have  $\bar{x} + 1 = 32 - x$ . And,  $32 - x$  is the additive inverse, which is what we wanted to compute.

# Negation by two's complement



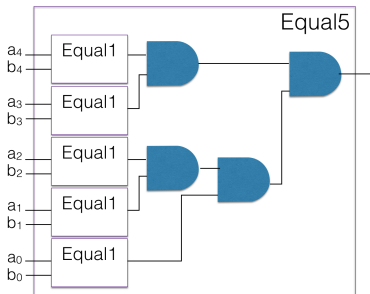
## IfThenElse5

Now that we can add and subtract 5-bit numbers, we might want to perform other operations on 5-bit numbers as well.



Depending on condition bit `c`, produces either 5-bit `t` or 5-bit `e`.

# Equal5



Takes two 5-bit values  $a_5a_4a_3a_2a_1a_0$  and  $b_5b_4b_3b_2b_1b_0$ . Returns **T** if and only if  $a_i == b_i$  for all  $i$ .